Pattern Recognition and Machine Learning

Chapter 5: Neural Networks

Reading Notes

August 18, 2026

Table of Contents

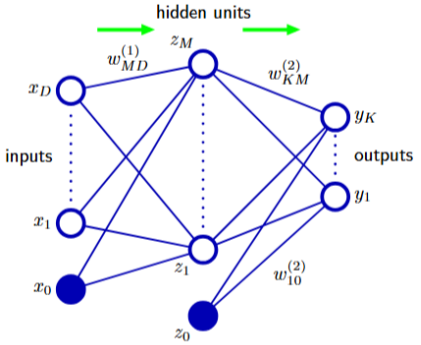
1. Feed-forward Network Functions
2. Network Training
3. Error Backpropagation
4. Regularization
5. Mixture Density Networks
6. Bayesian Neural Networks

Neural Network as Adaptive Basis Functions

Motivation: Linear models with fixed basis functions suffer from the curse of dimensionality. Neural networks make basis functions adaptive by parameterizing them.

Two-layer network (with logistic sigmoid output):

$$y_k(\mathbf{x}, \mathbf{w}) = \sigma\left(\sum_{j=1}^{M} w_{kj}^{(2)} h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)} \right) + w_{k0}^{(2)} \right)$$



Layer definitions:

- Input: $x_0, \ldots, x_D$ ($x_0 = 1$ for bias)
- Hidden: $z_j = h(a_j)$, $a_j = \sum_i w_{ji}^{(1)} x_i$
- Output: $y_k = \sigma(a_k)$, $a_k = \sum_j w_{kj}^{(2)} z_j$

Universal Approximation

A two-layer network with sufficient hidden units can approximate any continuous function on a compact domain.

Activation Functions and Gradient Flow

Hidden layer activations:

- **Sigmoid / tanh**: Saturating; vanishing gradients — not recommended for deep networks.
- **ReLU** ($\max(0, a)$): Sparse, cheap, standard default for most deep architectures.
- **SiLU (Swish)**: Smooth, non-monotonic; often outperforms ReLU in deeper models.

Output layer activations — matched to the target distribution:

- **Linear**: Regression
- **Sigmoid**: Binary classification
- **Softmax**: Multi-class classification

Practical Choice

For hidden layers, start with ReLU; consider SiLU for deeper or more complex tasks. Always pair output activation with the likelihood.

Error Functions from Maximum Likelihood

These are the negative log-likelihoods for each task type:

Regression (Gaussian target):

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\left(y(\mathbf{x}_n, \mathbf{w}) - t_n\right)^2$$

Binary classification (Bernoulli target):

$$E(\mathbf{w}) = -\sum_{n=1}^{N}\left[t_n \ln y_n + (1 - t_n)\ln(1 - y_n)\right]$$

Multi-class classification (Categorical target):

$$E(\mathbf{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{kn} \ln y_k(\mathbf{x}_n, \mathbf{w})$$

Key Pattern

$\frac{\partial E}{\partial a_k} = y_k - t_k$ — same for regression and classification.

Parameter Optimization

First-Order: Gradient Descent

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \cdot \mathbf{g}$$

Method	g	Batch Size
Batch GD	∇E	N
SGD	∇E_n	1
Mini-batch	$\frac{1}{B}\sum_{\mathcal{B}} \nabla E_n$	B

- N = dataset size, B = chosen batch size

Second-Order Methods

Newton-type update:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \mathbf{H}^{-1}\nabla E$$

Properties:

- Faster convergence
- Hessian $\mathbf{H}$: $O(W^2)$ to compute
- In practice: L-BFGS / conjugate gradients

Takeaway

First-order methods scale to large data; second-order methods converge faster but are computationally heavier.

Backpropagation Algorithm

Core Idea: Apply the chain rule recursively from output to input.

Define $\delta_j \equiv \partial E_n / \partial a_j$ (error signal for unit j).

Formula	Direction	Explanation
$\delta_k = y_k - t_k$	Start here	Error at output unit; measures prediction-target mismatch.
$\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$	Propagate $\leftarrow$	Backpropagate errors through weights; scale by local gradient.
$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i$	Local product	Multiply error signal by input activation to get weight gradient.

Chain rule: $\frac{\partial a_k}{\partial a_j} = w_{kj} h'(a_j) \;\Rightarrow\; \delta_j = \sum_k \delta_k \frac{\partial a_k}{\partial a_j} = h'(a_j) \sum_k w_{kj} \delta_k$

Algorithm Flow

Forward pass $\rightarrow$ output $\delta_k \rightarrow$ backpropagate $\delta_j \rightarrow$ compute gradients.

Jacobian and Hessian

Jacobian J: First-order sensitivity of outputs to inputs.

$$J_{ki} = \frac{\partial y_k}{\partial x_i}, \quad \mathbf{J} \in \mathbb{R}^{K \times D}$$

Neural net meaning: How much does output k change if input i is perturbed?

- Use: sensitivity analysis, input importance ranking, error propagation across modules.
- Compute: $O(W)$ per row via backpropagation.

Hessian H: Second-order curvature of the error surface.

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j}, \quad \mathbf{H} \in \mathbb{R}^{W \times W}$$

Neural net meaning: What is the local curvature of the loss landscape?

- Eigenvalues > 0 at a minimum; control learning rate (largest eigenvalue bounds η).
- Exact $O(W^2)$; outer product approx $\mathbf{H} \simeq \sum_n (\nabla y_n)(\nabla y_n)^\top$.

Regularization Overview

What: Techniques that reduce generalization error without hurting training fit.
Why: Deep networks overfit easily; regularization controls effective complexity.

Category	Method	Core Idea
Weight	L2 (weight decay)	$\tilde{E} = E + \frac{\lambda}{2}\|\mathbf{w}\|^2$; Gaussian prior, shrinks weights
Weight	L1 (lasso)	$\tilde{E} = E + \lambda\|\mathbf{w}\|_1$; Laplace prior, yields sparsity
Architecture	Dropout	Randomly drop units during training
Architecture	Batch Normalization	Normalize layer inputs; stabilizes training
Architecture	Weight sharing / CNNs	Built-in inductive bias; fewer parameters
Training	Data augmentation	Transform inputs (rotation, crop, flip)
Training	Early stopping	Stop when validation error worsens
Training	Noise injection	Add noise to inputs or weights
Training	Label smoothing	Softens one-hot targets; prevents overconfidence

Convolutional Neural Networks (CNNs)

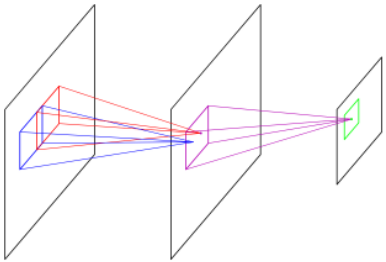
Three Key Mechanisms

- **Local receptive fields**: units see only a small spatial region.
- **Weight sharing**: same weights across positions $\Rightarrow$ translation invariance.
- **Subsampling (pooling)**: reduces resolution $\Rightarrow$ local robustness to shifts.

Typical Architecture

- Multiple conv + pooling layers
- Feature maps increase with depth
- Final fully connected + softmax

Inductive bias: spatial locality



Key Advantages

- Fewer parameters than fully connected
- Built-in translation invariance
- Hierarchical feature learning
- State-of-the-art on vision tasks

Mixture Density Networks (MDN)

Problem: Inverse problems often have multiple solutions — the conditional $p(t|x)$ is multimodal. Standard least-squares averages over modes, giving poor predictions.

MDN Solution: Output the parameters of a Gaussian mixture:

$$p(t|x) = \sum_{k=1}^{K} \pi_k(x)\, \mathcal{N}\big(t \mid \mu_k(x), \sigma_k^2(x)\big)$$

- Network outputs: π_k (mixing weights, via softmax), μ_k (means), σ_k (variances, via exp)
- Training: maximize log-likelihood
- Gradients computed by standard backpropagation

Key Advantage

Provides a **full conditional distribution**, not just a point estimate — captures multimodality and heteroscedasticity.

Why Bayesian Neural Networks?

Problem with standard training: We get a single weight vector (MLE or MAP).

- No idea how *confident* the model is
- Over-confident predictions far from training data

Bayesian solution: Instead of a single $\mathbf{w}$, maintain a **distribution** over weights.

$$p(\mathbf{w}|\mathcal{D}) \propto p(\mathcal{D}|\mathbf{w})\, p(\mathbf{w})$$

- Predict by *averaging* over all plausible weights:

$$p(t|\mathbf{x}, \mathcal{D}) = \int p(t|\mathbf{x}, \mathbf{w})\, p(\mathbf{w}|\mathcal{D})\, d\mathbf{w}$$

- Naturally gives **predictive uncertainty**

The Catch

Exact posterior is intractable for neural networks (nonlinearity + non-conjugacy). Need **approximate inference** — Laplace is one approach.

Laplace Approximation Framework

For regression, assume Gaussian noise and isotropic Gaussian prior:

$$p(t|x, w, \beta) = \mathcal{N}(t|y(x, w), \beta^{-1}), \quad p(w|\alpha) = \mathcal{N}(w|0, \alpha^{-1}I)$$

Posterior $p(w|\mathcal{D}, \alpha, \beta)$ is non-Gaussian; approximate by Gaussian around MAP w_{MAP}:

$$q(w|\mathcal{D}) = \mathcal{N}(w|w_{\mathrm{MAP}}, A^{-1}), \quad A = \alpha I + \beta H$$

where H is the Hessian of the sum-of-squares error.

Predictive distribution (linear approximation of network):

$$p(t|x, \mathcal{D}) \approx \mathcal{N}\left(t|y(x, w_{\mathrm{MAP}}), \beta^{-1} + g^{\top} A^{-1} g\right), \quad g = \nabla_w y(x, w_{\mathrm{MAP}})$$

Uncertainty Decomposition

Variance = noise variance + model uncertainty, the latter depending on input x.

Evidence Framework for Hyperparameters

Marginal likelihood (evidence) for α, β:

$$\ln p(\mathcal{D}|\alpha,\beta) \simeq -E(w_{\mathrm{MAP}}) - \frac{1}{2}\ln|A| + \frac{W}{2}\ln\alpha + \frac{N}{2}\ln\beta + \text{const}$$

with $E(w_{\mathrm{MAP}}) = \frac{\beta}{2}\sum_n (y_n - t_n)^2 + \frac{\alpha}{2} w_{\mathrm{MAP}}^\top w_{\mathrm{MAP}}$.

Re-estimation formulae:

$$\alpha = \frac{\gamma}{w_{\mathrm{MAP}}^\top w_{\mathrm{MAP}}}, \quad \frac{1}{\beta} = \frac{1}{N-\gamma}\sum_n (y_n - t_n)^2,$$

where $\gamma = \sum_i \frac{\lambda_i}{\alpha+\lambda_i}$ is the effective number of parameters, and λ_i are eigenvalues of βH.

Practical Procedure

Iterate: find w_{MAP} (MAP), compute γ, update α, β, repeat.

Bayesian Classification

For binary classification (logistic sigmoid output), no β; prior $p(w|\alpha) = \mathcal{N}(w|0, \alpha^{-1}I)$. MAP found by minimizing regularized cross-entropy:

$$E(w) = -\sum_n \{t_n \ln y_n + (1 - t_n) \ln(1 - y_n)\} + \frac{\alpha}{2} w^\top w.$$

Hessian H of negative log-likelihood; posterior approximated as $\mathcal{N}(w|w_{\mathrm{MAP}}, A^{-1})$ with $A = \alpha I + H$.

Predictive distribution (marginalising over w): Approximate output activation $a(x, w)$ as linear around w_{MAP}:

$$p(a|x, \mathcal{D}) = \mathcal{N}(a|a_{\mathrm{MAP}}, \sigma_a^2), \quad \sigma_a^2 = g^\top A^{-1} g, \quad g = \nabla_w a(x, w_{\mathrm{MAP}})$$

Then $p(t = 1|x, \mathcal{D}) = \int \sigma(a) p(a) da \approx \sigma\left(\kappa(\sigma_a^2)\, a_{\mathrm{MAP}}\right)$, with $\kappa(\sigma^2) = (1 + \pi\sigma^2/8)^{-1/2}$.

Effect

Marginalization pulls predictive probabilities towards 0.5, reflecting uncertainty.

Summary

- **Feed-forward Networks**: Adaptive basis functions; universal approximators
- **Training**: Likelihood-based errors; gradient descent & second-order methods
- **Backpropagation**: Efficient $O(W)$ gradient computation via chain rule
- **Regularization**: L2, L1, Dropout, BatchNorm, augmentation, early stopping
- **Convolutional Networks**: Local receptive fields, weight sharing, pooling
- **Mixture Density Networks**: Multimodal conditional densities
- **Bayesian Neural Networks**: Predictive uncertainty; Laplace approximation

Key Takeaway

Neural networks: flexible supervised learning.
Modern practice: **ReLU/SiLU + Adam + Dropout/BatchNorm + augmentation**.